Comprehension

Thursday, September 18, 2025 11:00 PM

Comprehension:

'''it is basically a phenomenon of creating a new collection by reducing the no.of instruction'''

or

using a single instruction we can create collection

Types of comprehension:

1.list comprehension
2.set comprehension
3.dictionary comprehension

Note: we cann't use comprehension for string and tuple because those are immutable data types

list comprehension:

it is a phenomenon of creating a new list from existed collection

syntaxes:

```
1.variablename=[value/variable(Tsb) for variablename in collection  if condition]
                                                    #or
2.variablename=[Tsb if condition else Fsb for var_name in collection]
                                                    #or
3.variablename=[variable1,variable12 or i,j for variable1 in collection1 for variable2 in collection2 ]
```

1.wap program to create a list consiste of 1 to 25 integer values

```
out=[]
for i in range(1,26):
    out.append(i)
print(out)
#out=[i for i in range(1,26)]
print([i for i in range(1,26)])
```

2.wap to create a list consist of square of each and every integer between the range 1 to 20 only if it is mutliple of 3

```
res=[]
for i in range(1,21):
    if i%3==0:
```

```
        res.append(i**2)
    print(res)

    #by using list comprehension

    res=[i**2 for i in range(1,21)  if i%3==0 ]
    print(res)

    #or
    print([i**2 for i in range(1,21)  if i%3==0 ])
```

3.wap to extract strings from the given list

```
    l=[12,3.4,'python',8+9j,False,'abcd','students']
    k=[ i for i in l if type(i)==str]
    print(k)
```

4.wap to get the following output

```
    s='programs on comprehension'
    output=['programs','on','cn']
    Notes:
       if the length of word is even you should store as it is
       or else print starting and ending character
```

5.wap to store square of integer if it is even else store cube of the same number
range(1,30)

#wap to get the following output
col1='AB'
col2=[1,2,3]
[('A',1)('A',2),('A',3),('B',1),('B',2),('B',3)]

Assignment – 29/08/25

1.Write a program to find the square root of all the integers between 10 and 100.
2.Write a program to find the factorial of all the integers present in a given list.
3.Write a program to extract all the complex numbers from the given set.
Note:for above question take your own Input.
4.Write a program to get the following output:
Input: "Python is a Sython"
Output: {'PYTHON', 'si', 'a', 'SYTHON'}
Note:
If a word in the string starts with an uppercase letter, convert the entire word to uppercase.
Otherwise, store the reverse of that word.
(Hint: use split() to separate words).
write The program for all above Questions by using both list and set comprehension

Wap to extract integers and find the square of them from the given collection.

```
d=[10,'holiday',20,'Over',25,'Sunday',30]
s=[i**2 for i in d if type(i)==int]
print(s)

k=[x.upper() for i in d if type(i)==str]
print(k)


data=[10,['holiday',20],('Over',25),'Sunday',30]
n=[j**2 for i in data for j in (i if type(i) in [str,list,tuple,set,dict] else [i] )if type(j)==int]
print(n)

#wap to to extract strings from this below collection
    data=[['holiday',20],('Over',25),{'Sunday',30}]
    s=[j for i in data for j in i if type(j)==int]
    print(s)
#wap to extract names from the given collection
    n=[{'roll':1,'name':'Kavi'},{'roll':2,'name':'pavi'},'Python',50]
    s=[i['name'] for i in n if type(i)==dict]
    print(s)
    n=[{1,2,3},{4,5,6,},{2,4}]
    f=[j for i in n for j in i]
    print(f)

#wap to extract only even numbers from the given collection
    #n=[{1,2,3},{4,5,6,},{2,4}]
    n=[[10,40],50,28,(43,33,40)]
    k={j for i in n for j in(i if type(i) in [list,tuple,set] else [i])}#[10,28,33,40,43,50]
    print(k)
    s=sorted([j for i in n for j in(i if type(i) in [list,tuple,set] else [i])])[-2]
    print(s)



#find second max number
    n=[[10,40],50,28,(43,33,43,40)]

    k=[j for i in n for j in(i if type(i) in [list,tuple,set] else [i])]#[10,28,33,40,43,50]
    print(k)
    s=sorted([j for i in n for j in(i if type(i) in [list,tuple,set] else [i])])[-2]
    print(s)
    k={j for i in n for j in(i if type(i) in [list,tuple,set] else [i])}
    #print(k)
    print(sorted(k)[-2])


#Assignement--->29/08/25
```

1.wap to find the squareroot of all the integers between 10 to 20

2.wap to find the factorial of all the integers present in a given list

```
def fact(n):
    if n==1 or n==0:
        return 1
    return n*fact(n-1)
#print(fact([10,20,30]))
print([fact(i) for i in eval(input('enter the list:')) if type(i)==int])
#or
import math
print(math.factorial(i) for i in eval(input('enter the list:')) if type(i)==int])
```

Syntax of Factorial()--->import math
math.factorial(value)'''

3.wap to extract all the complex numbers from the given set
4.wap program to get the following output
input:'Python is a Sython'
output:{'PYTHON','si','a','SYTHON'}
Note:In above question if the string is started with Uppercase convert entire string Upper else store the reverse string of that Word(use split())

#set comprehension

it is a phenomenon of creating new set

syntaxes:
1.variable_name={value for variable_name in collection if condition}
#or
2.variable_name={tsb if condition else fsb for variable in collection}
#or
3.variable_name={i,j or variable1,variable2,for variable1 in collection for variable2 in collection}

1.wap to find the squareroot of all the integers between 10 to 20

```
import math
n=[math.sqrt(i) for i in range(10,21)]
print(n)
#or
print([i**0.5 for i in range(10,21)])
```

2..wap to extract all the complex numbers from the given set

n={10,35,5+6j,'hi',1+2j,0j,True}

```
out={i for i in n if type(i)==complex}
print(out)
#nested collection:

n={10,35,5+6j,'hi',1+2j,0j,True,(8+6j,200,7+4j)}
k={j for i in n for j in(i if type(i) in [list,tuple,set] else [i])if type(j)==complex}
print(k)
```

3.wap program to get the following output

```
#input:'Python is a Sython'
#output:{'PYTHON','si','a','SYTHON'}

    s='Python is a Sython hai Heloo'.split()#['Python','is','a','Synthon']
    n={i.upper() if 'A'<=i[0]<='Z' else i[::-1] for i in s}
    print(n)
```

zip():it is a inbuilt function which is used to traverse through the multiple collections.

```
syntax:
for variabl1,variable2...variablen in zip(collection1,collection2....collection n)
```

Dictionary Comprehension:

it is phenomemnon of creating new dictionary

```
Syntax:
   1.variable_name={Key:value for variable_name in collection if condition}
   2.variable_name={Key:value for variable1,variable2.. in zip(collection1,collection2..) if condition}

   3.variable_name={Key:val1 if condition else value2 for variable1,variable2.. in
zip(collection1,collection2..) }
```

wap to get the following output

```
#output:{1:1,2:4,3:9,4:16,5:25}

        print({i:i**2 for i in range(1,6)})
```

wap to get the following output

```
s='Hai hello howw are youu'
#output={'howw':4,'youu':4}

    n={i:len(i) for i in s.split() if len(i)%2==0}
        #or
    print({i:len(i) for i in 'Hai hello howw are youu'.split() if len(i)%2==0})
```

```
#wap to map two list collections in the form of dictionary
l1=['a','b','c',(7,8)]
l2=[10,20,30]
#Output:{'a':10,'b':20.....}

    n={i:j for i,j in zip(l1,l2) if type(i) not in [list,set,dict]}
    print(n)
    #or
    print({i:j for i,j in zip(l1,l2) if type(i) not in [list,set,dict]})


#Assignment--->30/08/2025
#1.l=['abcd','nayan','data','aba']
Output:{'abcd':'dcba','nayan':5....}
note:if the string is pallindrome store that as a key and store length of that string
as a value or else store reverse of that string in value position
#2.wap to get the following output
range(1,11)
Output:{1:1,2:2,3:3,4:4,5:25,6:6,7:7,8:8,9:9,10:100}
Note:store the all values in key position,but in place value store
square of that key if that key divisible 5 or else store as it is(key:key)


#wap program to extract a strings
data=[10,['holiiday',20],('over',25),'sunday',30]
n=[j for i in data for j in(i if type(i) in [str,list,tuple,dict] else i)if type(j)==int]
print(n)

#by using normal way
for i in [10,['holiiday',20],('over',25),'sunday',30]:
   for j in [i]:
      if type(j)==int:
         print(j)


#Assignment--->30/08/2025
#1.l=['abcd','nayan','data','aba']
#Output:{'abcd':'dcba','nayan':5....}
#note:if the string is pallindrome store that as a key and store length of that string
#as a value or else store reverse of that string in value position
n={i:len(i) if i==i[::-1] else i[::-1] for i in ['abcd','nayan','data','aba']}
print(n)
'''
2.wap to get the following output
range(1,11)
Output:{1:1,2:2,3:3,4:4,5:25,6:6,7:7,8:8,9:9,10:100}

Note:store the all values in key position,but in place value store
```

square of that key if that key divisible 5 or else store as it is(key:key)

```
n={i:i**2 if i%5==0 else i for i in range(1,50)}
print(n)
```

'Wap to to print 'Pass' if the students got more than 35 else false in dictionary format store students names as a key and store result as a value.

```
col1=['parshivi','Sanjeev','Davya','Nikin']
col2=[85,40,72,33]
print({i:'pass' if j>35 else 'fail' for i,j in zip(col1,col2 )})
Output:{'parshivi':'Pass','Sanjeev':'Pass','Nikin':'Fail'}
```

2.Wap program to print Grade of Each students based on their marks

```
if marks>=80-->'A'
marks>=50--->'B'
else 'C'
#col1=['parshivi','sanjeev','davya','nikin']
#col2=[85,40,72,33]
print({i:'A' if j>=80 else 'B' if j>=50 else 'c' for i,j in zip(['parshivi','sanjeev','davya','nikin'],[85,40,72,33])})
```

Assignment Questions--->1-09-25

3.Wap to print salary classification like low salary or High salary

```
Input--->employee=['parshvi','sanjeev','davya','nikin','Prashanth','Pavithra','Srinivas']
                emp_sal=[50000,30000,25000,20000,45000,60000,10000]
Output:-{'parshvi':'High Salary','sanjeev':'Low Salary'.....}
if sal>=40000--->'High salary'
or else 'Low salary'
```

4.Wap to print students attendence

```
students=['parshvi','sanjeev','davya','nikin','Prashanth','Pavithra','Srinivas']
attendence=[1,0,2,1,0,3,4,0]#
Note:Present=1,Absent=0
Output:{'parshvi:Present','sanjeev':'Absent'.....}
```

5.Print student present if the student active or Body Present mind Apsent

```
students=['parshvi','sanjeev','davya','nikin','Prashanth','Pavithra','Srinivas']
attendence=['Active','Not active','Active','Not active','Active','Not active','Active']
```

6.Print 'No supply' student having count 0 or 'BAckbencher'(Supply7

```
students=['parshvi','sanjeev','davya','nikin','Prashanth','Pavithra','Srinivas']
Supply=[1,0,2,1,0,3,4,0]
```

5.wap program to print student or sleeping in the morining class or not

```
students=['parshvi','sanjeev','davya','nikin','Prashanth','Pavithra','Srinivas']
students_eyes=['Open','Closed','Open','Open','Closed','Open','Close']
```

Note :if students eyes are closed They are in body present Brain Absent (print Sleeping) or else Print 'Awake'

Q&A: Comprehensions in Python

1. List Comprehension (LC)

Q1. What is list comprehension?

It is a concise way to create lists using a single line of code.

```
nums = [x for x in range(5)]
print(nums)  # [0, 1, 2, 3, 4]
```

Q2. How is list comprehension different from using a for loop?

- Loop: requires multiple lines.
- LC: shorter and often faster.

```
# Using loop
squares = []
for x in range(5):
    squares.append(x**2)
# Using LC
squares = [x**2 for x in range(5)]
```

Q3. Can we use if condition in list comprehension?

Yes, to filter elements.

```
nums = [x for x in range(10) if x % 2 == 0]
print(nums)  # [0, 2, 4, 6, 8]
```

Q4. Can we use if-else in list comprehension?

Yes, but syntax is slightly different.

```
nums = ["Even" if x % 2 == 0 else "Odd" for x in range(5)]
print(nums)  # ['Even', 'Odd', 'Even', 'Odd', 'Even']
```

Q5. Can list comprehension have nested loops?

Yes.

```
pairs = [(x, y) for x in [1, 2] for y in [3, 4]]
print(pairs)  # [(1, 3), (1, 4), (2, 3), (2, 4)]
```

Q6. Is list comprehension faster than a for loop?

Usually yes, because it is optimized internally.

Q7. Can we call functions inside a list comprehension?

Yes.

```
def square(n): return n*n
nums = [square(x) for x in range(5)]
print(nums)  # [0, 1, 4, 9, 16]
```

Q8. How to flatten a nested list using list comprehension?

```
nested = [[1, 2], [3, 4], [5, 6]]
flat = [x for sub in nested for x in sub]
print(flat)  # [1, 2, 3, 4, 5, 6]
```

Q9. Can list comprehension be used with strings?
Yes.

```
word = "python"
letters = [ch.upper() for ch in word]
print(letters)  # ['P', 'Y', 'T', 'H', 'O', 'N']
```

Q10. What happens if the iterable is empty?
It produces an empty list.

```
print([x for x in []])  # []
```

2. Set Comprehension (SC)
Q1. What is set comprehension?
It is a compact way to create sets.

```
nums = {x for x in range(5)}
print(nums)  # {0, 1, 2, 3, 4}
```

Q2. How is set comprehension different from list comprehension?
- LC → creates a list (ordered, allows duplicates).
- SC → creates a set (unordered, unique values).

Q3. Can we remove duplicates automatically in set comprehension?
Yes, sets automatically remove duplicates.

```
nums = {x%3 for x in range(10)}
print(nums)  # {0, 1, 2}
```

Q4. Can we use if-else conditions in set comprehension?
Yes.

```
nums = {x if x%2==0 else -x for x in range(5)}
print(nums)  # {0, 2, -1, -3, 4}
```

Q5. Can set comprehension contain nested loops?
Yes.

```
pairs = {(x, y) for x in [1, 2] for y in [3, 4]}
print(pairs)  # {(1, 3), (1, 4), (2, 3), (2, 4)}
```

Q6. Does set comprehension maintain order?
No, sets are unordered collections.

Q7. Can set comprehension be used for mathematical operations?
Yes.

```
squares = {x**2 for x in range(5)}
print(squares)  # {0, 1, 4, 9, 16}
```

Q8. Can set comprehension generate an empty set?
No, {} creates an empty dictionary. Use set().

```
empty_set = set()
print(empty_set)  # set()
```

Q9. What happens if duplicate elements are generated inside a set comprehension?
They are automatically removed.

```
nums = {x//2 for x in range(6)}
print(nums)  # {0, 1, 2, 3}
```

Q10. Is set comprehension faster than creating sets with a loop?
Usually yes, because it is optimized.

3. Dictionary Comprehension (DC)
Q1. What is dictionary comprehension?
It is a compact way to create dictionaries.

```
squares = {x: x**2 for x in range(5)}
print(squares)  # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Q2. How is dictionary comprehension different from set or list comprehension?
- DC → produces key-value pairs.
- LC/SC → produce elements only.

Q3. Can we use if condition in dictionary comprehension?
Yes.

```
evens = {x: x**2 for x in range(6) if x%2==0}
print(evens)  # {0: 0, 2: 4, 4: 16}
```

Q4. Can we use if-else inside dictionary comprehension?
Yes.

```
nums = {x: ("Even" if x%2==0 else "Odd") for x in range(5)}
print(nums)  # {0: 'Even', 1: 'Odd', 2: 'Even', 3: 'Odd', 4: 'Even'}
```

Q5. How to swap keys and values using dictionary comprehension?

```
data = {"a": 1, "b": 2, "c": 3}
swapped = {v: k for k, v in data.items()}
print(swapped)  # {1: 'a', 2: 'b', 3: 'c'}
```

Q6. Can dictionary comprehension be used with nested loops?
Yes.

```
pairs = {(x, y): x+y for x in [1, 2] for y in [3, 4]}
print(pairs)  # {(1, 3): 4, (1, 4): 5, (2, 3): 5, (2, 4): 6}
```

Q7. What happens if two keys are the same in dictionary comprehension?
The last value will overwrite the previous one.

```
nums = {x%2: x for x in range(5)}
print(nums)  # {1: 3, 0: 4}
```

Q8. Can dictionary comprehension call functions while creating values?
Yes.

```
def cube(n): return n**3
cubes = {x: cube(x) for x in range(4)}
print(cubes)  # {0: 0, 1: 1, 2: 8, 3: 27}
```

Q9. Is dictionary comprehension faster than using a normal loop?
Yes, generally more efficient.

Q10. Can dictionary comprehension generate nested dictionaries?
Yes.

```
nested = {x: {y: y**2 for y in range(3)} for x in range(2)}
print(nested)
# {0: {0: 0, 1: 1, 2: 4}, 1: {0: 0, 1: 1, 2: 4}}
```